❹ Flow to scan lines

This source contains a single `ByteString`, which is then parsed. The `CsvParsing .lineScanner()` is a `Flow[ByteString,List[ByteString],NotUsed]`. A list of words is created for each line break in the `ByteString`. Since it's a flow, you apply it as a `.via`.

The next listing shows the rest of the previous test. To check the result, wait until the `Future` is complete.

Listing 14.6 Scanning a CSV source

```
class CSVSpec
    extends AnyFlatSpec
    with Matchers
    with BeforeAndAfterAll {

  implicit val system = ...                                                    ❶

  implicit val ec: ExecutionContext = ExecutionContext.Implicits.global        ❷

  "A CSV Source" should "be readable" in {

    val result: Future[Seq[List[ByteString]]] = ...                            ❸

    result.onComplete {                                                        ❹
      case Success(lines) =>                                                   ❹
        lines.head should be(                                                  ❹
          List(
            ByteString("Alice"),
            ByteString("Pleasence ")))

        lines(1) should be(
          List(
            ByteString("Bob"),
            ByteString("Marley"),
            ByteString("Jung")))
    }
  }
```